

NGÀY 1: LÀM QUEN VỚI RUST

Bài 1: Tại sao RUST ra đời

Nếu bạn đã biết C/C++, Python hoặc JavaScript thì Rust mang đến một thế giới hoàn toàn khác biệt, Rust giải quyết bài toán hóc búa nhất của lập trình hệ thống: Làm sao để vừa nhanh như C/C++ nhưng lại vừa an toàn tuyệt đối về bộ nhớ (memory safety) mà không cần bộ thu gom rác (garbage collector) như Java/Python?

Rust ra đời với ba “vũ khí tối thượng”:

- **Hiệu năng vượt trội:** Biên dịch trực tiếp ra mã máy, không có runtime nặng nề.
- **An toàn tuyệt đối:** Trình biên dịch của Rust (rustc) cực kỳ khó tính. Nó sẽ từ chối biên dịch nếu mã nguồn của bạn có nguy cơ rò rỉ bộ nhớ (memory leak) hoặc xung đột dữ liệu (data race).
- **Công cụ đi kèm:** Trình quản lý gói **cargo** của Rust được đánh giá là một trong những công cụ quản lý dự án tốt nhất thế giới hiện tại.

Bài 2: Cài đặt môi trường

Cách tốt nhất để cài đặt Rust là sử dụng **rustup**.

Trên Linux/macOS:

Chạy lệnh sau trong Terminal:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

Trên Windows:

1. Tải file cài đặt rustup-init.exe tại trang chủ <https://rustup.rs/>
2. Chạy file và làm theo hướng dẫn trên màn hình. Lưu ý: Rust yêu cầu Build Tools cho Visual Studio, trình cài đặt sẽ hướng dẫn bạn cài đặt nếu chưa có.
3. Kiểm tra cài đặt: Sau khi cài đặt xong, hãy mở lại Terminal/Command Prompt và gõ:

```
rustc --version  
cargo --version
```

Phần cài đặt chung:

- Sau khi cài đặt thành công rustup, ta tiến hành cài đặt extension trên VS Code:
 - rust-analyser (bắt buộc)
 - CodeLLDB (tùy chọn): dùng để debug sau này.

Bài 3: Viết chương trình đơn giản với Rust

Bước 1: Tạo một file có tên là main.rs (.rs là phần mở rộng của file nguồn Rust).

Bước 2: Viết mã:

```
fn main() {  
    println!("Hello, World!");  
}
```

Giải thích nhanh:

- `fn main()`: Đây là hàm khởi chạy (entry point) của mọi chương trình Rust. Khi chạy chương trình, Rust sẽ tìm đến hàm này đầu tiên.
- `println!(...)`: Đây là một Macro (nhận biết nhờ dấu chấm than), không phải là hàm thông thường. Nó dùng để in dữ liệu ra màn hình.
- Tương tự như C/C++ hay JavaScript, mọi câu lệnh thực thi trong Rust bắt buộc phải kết thúc bằng dấu chấm phẩy.

Bước 3: Biên dịch và chạy chương trình.

Mở Terminal tại thư mục chứa file main.rs và chạy lệnh thực thi sau:

```
rustc main.rs
```

Lệnh này sẽ tạo ra một file thực thi (file nhị phân), tên là main (Linux/macOS) hoặc main.exe (Windows).

Chạy file đó:

```
./main # Trên Linux/macOS  
.\main.exe # Trên Windows
```

Màn hình sẽ hiển thị: "Hello, World!".

Bài 4: Làm quen với Cargo

Trong thực tế, bạn sẽ hiếm khi phải dùng lệnh `rustc` trực tiếp. Thay vào đó, bạn sẽ dùng Cargo - hệ thống quản lý dự án và build quản lý của Rust.

Hãy tạo một dự án mới bằng Cargo bằng lệnh Terminal sau:

```
cargo new hello_cargo  
cd hello_cargo
```

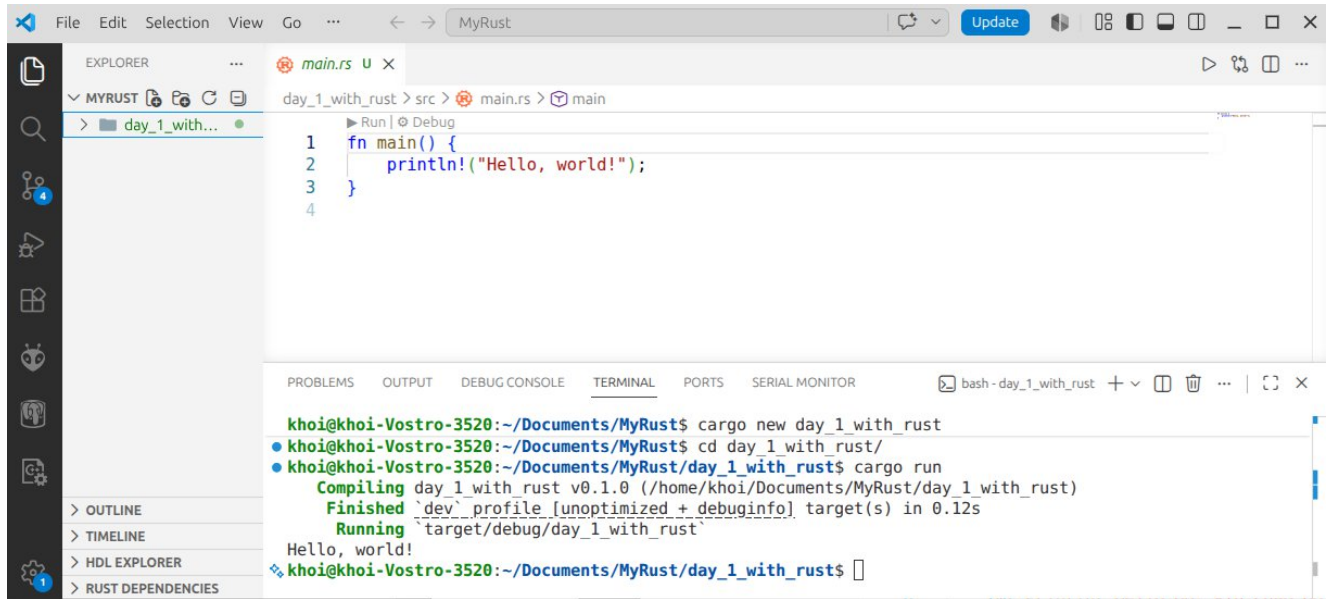
Cargo sẽ tự động tạo cho bạn một thư mục dự án với cấu trúc tiêu chuẩn:

- **Cargo.toml**: File cấu hình dự án (tên dự án, phiên bản, các thư viện bên ngoài cần dùng).
- **src/main.rs**: File chứa mã nguồn chính (đã có sẵn code Hello World).

Để chạy dự án với Cargo, bạn chỉ cần một lệnh duy nhất:

```
cargo run
```

Cargo sẽ tự động biên dịch (nếu có thay đổi) và chạy chương trình ngay lập tức.



The screenshot shows an IDE window titled "MyRust". The Explorer panel on the left shows a project structure with "MYRUST" and a subdirectory "day_1_with_rust". The main editor displays the file "main.rs" with the following code:

```
1 fn main() {  
2     println!("Hello, world!");  
3 }  
4
```

The bottom panel shows the "TERMINAL" output with the following commands and results:

```
khoi@khai-Vostro-3520:~/Documents/MyRust$ cargo new day_1_with_rust  
khoi@khai-Vostro-3520:~/Documents/MyRust$ cd day_1_with_rust/  
khoi@khai-Vostro-3520:~/Documents/MyRust/day_1_with_rust$ cargo run  
Compiling day_1_with_rust v0.1.0 (/home/khai/Documents/MyRust/day_1_with_rust)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s  
Running `target/debug/day_1_with_rust`  
Hello, world!  
khoi@khai-Vostro-3520:~/Documents/MyRust/day_1_with_rust$
```

NGÀY 2: CÚ PHÁP CỐT LÕI VÀ HỆ THỐNG KIỂU DỮ LIỆU

Hôm nay, chúng ta sẽ học cách giao tiếp với Rust thông qua các biến số, các kiểu dữ liệu cơ bản và cách điều khiển luồng chương trình.

Bài 1: Làm quen với các macro cơ bản

Trong Rust, bạn sẽ thấy các lệnh in đều có dấu chấm than (!) ở cuối. Đây không phải là hàm thông thường mà là các macro. Rust cung cấp một bộ macro in ấn mạnh mẽ và an toàn giúp bạn xuất dữ liệu ra màn hình.

Dưới đây là 4 macro bạn sẽ dùng nhiều nhất:

1. `print!` và `println!`

- `print!`: In dữ liệu ra màn hình nhưng không xuống dòng.
- `println!`: Tương tự như `print!` nhưng có ký tự xuống dòng.

Để in giá trị của một biến hoặc một biểu thức, Rust sử dụng cặp ngoặc nhọn làm ký tự giữ chỗ (placeholder).

```
fn main() {  
    let name = "Rust";  
    let version = 2026;  
  
    // Cách 1: Truyền biến vào sau chuỗi  
    println!("Welcome to {} version {}", name, version);  
  
    // Cách 2: Truyền trực tiếp  
    println!("Welcome to {name} version {version}");  
  
    // Cách 3: Định vị bằng số thứ tự (index)  
    println!("Welcome to {0} version {1}. {0} is easy to learn.", name,  
version);  
}
```

- **Finished** `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
Running `target/debug/examples/02\_print`
Welcome to Rust version 2026
Welcome to Rust version 2026
Welcome to Rust version 2026. Rust is easy to learn.
* Terminal will be reused by tasks, press any key to close it.

Nâng cao: Bạn có thể điều chỉnh cách hiển thị của dữ liệu ngay trong cặp ngoặc nhọn:

```
fn main() {
```

```
println!("Pi={:.2}", 3.1415);

let num = 45;
println!("Number {0} in binary is {0:b}", num);
println!("Number {0} in hex is {0:x}", num);
}
```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.09s
Running `target/debug/examples/02\_print\_advance`
Pi=3.14
Number 45 in binary is 101101
Number 45 in hex is 2d
* Terminal will be reused by tasks, press any key to close it.

2. eprint! và eprintln!

Khi viết các chương trình thực tế, ngoài màn hình hiển thị thông thường (stdout), máy tính còn có một kênh riêng để hiển thị lỗi (stderr).

Khi này ta nên sử dụng eprint! hoặc eprintln! khi in ra các thông báo lỗi hoặc cảnh báo. Nó giúp hệ thống tách biệt luồng dữ liệu sạch và luồng dữ liệu lỗi.

3. dbg!

Khi bạn muốn biết một biến hoặc một biểu thức đang có giá trị là gì mà không muốn viết cả câu println! dài dòng, hãy dùng dbg!

```
fn main() {
    let a = 5;
    let _b = dbg!(a * 10);
}
```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
Running `target/debug/examples/02\_dbg`
[examples/02_dbg.rs:5:14] a * 10 = 50
* Terminal will be reused by tasks, press any key to close it.

Lưu ý quan trọng: Không phải dữ liệu nào cũng có thể in ra bằng {}. Đối với các kiểu phức tạp hơn (như mảng, cấu trúc struct), bạn phải dùng {:?} (gọi là debug format) để in.

```
fn main() {
    let arr = [1, 2, 3];

    // Normal style
    println!("Normal: My array is {:?}", arr);

    // Fancy style
    println!("Fancy: My array is {:#?}", arr);
}
```

```
}
```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
Running `target/debug/examples/02\_print\_special\_type`
Normal: My array is [1, 2, 3]
Fancy: My array is [
 1,
 2,
 3,
]
* Terminal will be reused by tasks, press any key to close it.

Bài 2: Biến số và Tính bất biến (Immutability)

Trong hầu hết các ngôn ngữ khác, biến số có thể thay đổi giá trị thoải mái. Nhưng với Rust, mặc định mọi biến số đều là bất biến. Nghĩa là một khi đã gán giá trị, bạn không thể thay đổi nó.

Hãy thử đoạn code sau trong hàm main():

```
fn main() {  
    let x = 5;  
    println!("x= {x}");  
    x = 6; // <= Dòng này sẽ bị lỗi khi biên dịch!  
}
```

Tại sao Rust lại làm vậy?

Trả lời: Để đảm bảo an toàn và tối ưu hiệu năng. Nếu bạn muốn biến đó có thể thay đổi được, bạn phải thêm từ khóa **mut** (mutable) một cách tường minh:

```
fn main() {  
    let mut x = 5;  
    println!("x= {x}");  
    x = 6;  
    println!("x= {x}");  
}
```

- * Executing task: cargo run --package TutorialRust --example 02_mut

 Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
 Running `target/debug/examples/02\_mut`
x= 5
x= 6
* Terminal will be reused by tasks, press any key to close it.

Hằng số (Constants)

Giống như tính bất biến, hằng số (const) không thể thay đổi. Tuy nhiên, hằng số bắt buộc phải khai báo kiểu dữ liệu và có đặt ở phạm vi toàn cục (bên ngoài hàm main).

```
const SECONDS_A_DAY: u32 = 24 * 60 * 60;

fn main() {
    println!("A day will have {SECONDS_A_DAY} seconds!");
}
```

```
* Executing task: cargo run --package TutorialRust --example 02_const

Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s
Running `target/debug/examples/02_const`
A day will have 86400 seconds!
* Terminal will be reused by tasks, press any key to close it.
```

Bài 3: Các kiểu dữ liệu cơ bản

Rust là một ngôn ngữ tĩnh (statically typed), nghĩa là nó phải biết kiểu dữ liệu của tất cả các biến tại thời điểm biên dịch. Thường thì Rust sẽ tự suy đoán kiểu dữ liệu, nhưng đôi khi bạn cần định nghĩa rõ ràng bằng dấu hai chấm. Ví dụ: Với ví dụ về hằng số ở trên, hằng số SECONDS_A_DAY được định nghĩa bởi kiểu dữ liệu u32, tương tự với unsigned 32-bit integer.

Cũng với ví dụ trên, nếu ta thay bằng kiểu u32 bằng u16 thì khi chạy ta sẽ nhận được lỗi tràn số:

```
examples > 02_lesson1b.rs SECONDS_A_DAY

1 // Constants
2 const SECONDS_A_DAY: u16 = 24 * 60 * 60;
3
  ▶ Run | ⚙ Debug
4 fn main() {
5     println!("A day will have {SECONDS_A_DAY} seconds!");
6 }

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

❶ khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_lesson1b
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
error[E0080]: attempt to compute `1440_u16 * 60_u16`, which would overflow
--> examples/02_lesson1b.rs:2:28
2 | const SECONDS_A_DAY: u16 = 24 * 60 * 60;
  |                               ~~~~~~ evaluation of `SECONDS_A_DAY` failed here

note: erroneous constant encountered
--> examples/02_lesson1b.rs:5:32
5 |     println!("A day will have {SECONDS_A_DAY} seconds!");
  |                               ~~~~~~

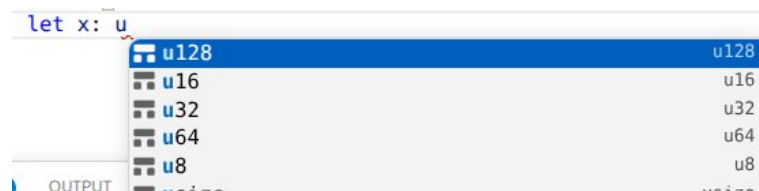
For more information about this error, try `rustc --explain E0080`.
error: could not compile `TutorialRust` (example "02_lesson1b") due to 1 previous error
```

Ta hãy cùng khám phá thêm về các kiểu dữ liệu có trong Rust nhé.

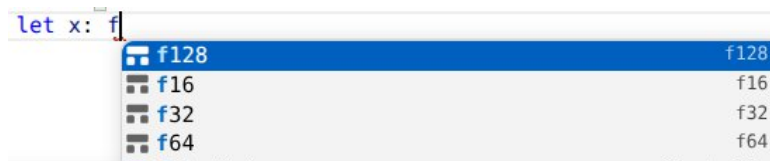
Kiểu vô hướng (Scalar Types):

Đại diện cho một giá trị đơn lẻ:

- **Số nguyên (Integer):** Chia làm hai loại: có dấu (i) và không dấu (u). Độ dài từ 8-bit đến 128-bit.

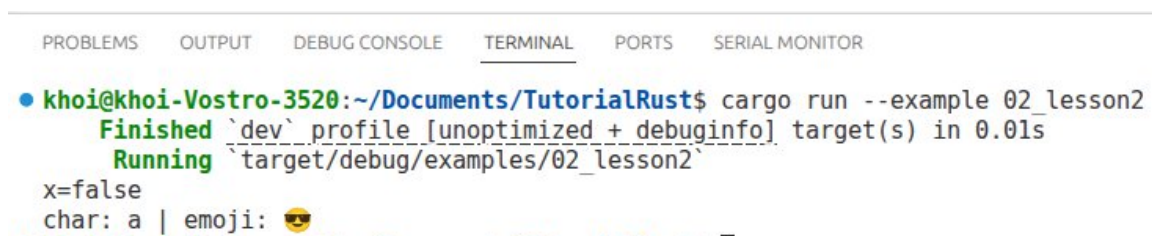


- **Số thực (Floating-point):** Độ dài từ 16-bit đến 128-bit.



- **Kiểu Logic (Boolean):** true hoặc false
- **Kiểu ký tự (Character):** char, bọc trong nháy đơn 'a'. Lưu ý: char trong Rust đại diện cho một giá trị Unicode (4 bytes), nên nó có thể chứa được cả biểu tượng cảm xúc (emoji) hoặc chữ tiếng Việt có dấu.

```
fn main() {  
    let x = false;  
    println!("x={x}");  
    let c: char = 'a';  
    let e: char = '😄';  
    println!("char: {c} | emoji: {e}");  
}
```



Kiểu dữ liệu phức hợp (Compound Types)

Gom nhiều giá trị thành một nhóm.

- **Tuple:** Gom nhiều giá trị có kiểu dữ liệu khác nhau thành một khối cố định như Python.

```
fn main() {
```



```
let tup: (i32, f64, char) = (500, 6.4, ' ');
let (x, y, z) = tup;
let five_hundred = tup.0; // Dùng dấu chấm để lấy giá trị tại index
println!("x={x} | y={y} | z={z} | five_hundred={five_hundred}");
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

```
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_tuple
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
Running `target/debug/examples/02_tuple`
x=500 | y=6.4 | z= | five_hundred=500
```

- **Mảng (Array):** Gom nhiều giá trị cùng kiểu dữ liệu. Mảng trong Rust có độ dài cố định.

```
fn main() {
    let a = [1, 2, 3, 4, 5, 6];
    let first = a[0];
    let second = a[1];
    let last = a[5];
    println!("first={first} | second={second} | last={last}");
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

```
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_array
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
Running `target/debug/examples/02_array`
first=1 | second=2 | last=6
```

Bài 4: Hàm (Functions)

Hàm trong Rust sử dụng từ khóa `fn`. Rust sử dụng kiểu viết `snake_case` (chữ thường, phân tách bằng dấu gạch dưới) để đặt tên hàm.

```
fn main() {
    let result = sum(5, 10);
    println!("Sum={result}");
}

fn sum(a: i32, b: i32) -> i32 {
    a + b // <= Không có dấu chấm phẩy ở đây!
}
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_function
  Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
  Running `target/debug/examples/02_function`
Sum=15
```

Lưu ý: Trong Rust, dòng `a + b` không có dấu chấm phẩy được gọi là một Expression (Biểu thức). Nó sẽ tự động trả về giá trị mà không cần từ khóa `return`. Nếu bạn thêm dấu chấm phẩy, nó sẽ biến thành một Statement (Câu lệnh) và trả về `gì cả` (gây lỗi biên dịch trong trường hợp này).

Bài 5: Cấu trúc điều khiển luồng (Control Flow)

Câu lệnh điều kiện `if - else`

Cơ bản giống các ngôn ngữ khác, nhưng điều kiện không cần bọc trong dấu ngoặc đơn.

```
fn main() {
    let num = 5;

    if num % 2 == 0 {
        println!("This is an even number!");
    } else {
        println!("This is an odd number!");
    }
}

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_if_else
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.01s
  Running `target/debug/examples/02_if_else`
This is an odd number!
❖ khoi@khoi-Vostro-3520:~/Documents/TutorialRust$
```

Đặc biệt, vì `if` là một biểu thức, bạn có thể dùng nó để gán giá trị:

```
let cond = true;
let number = if cond { 5 } else { 6 }; // hai nhánh phải cùng kiểu dữ liệu
println!("number={number}");

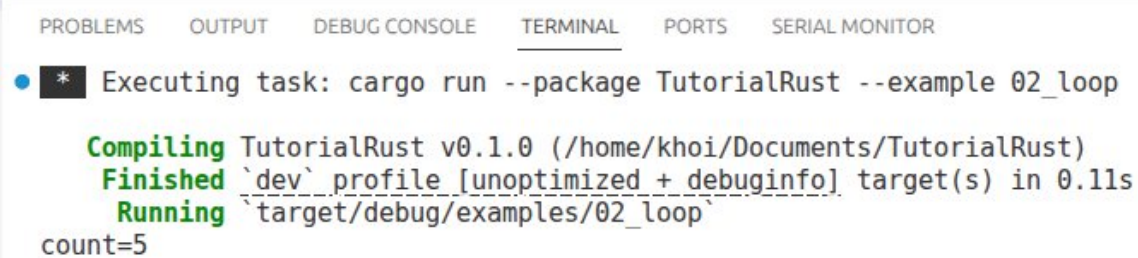
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo run --example 02_if_else
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.00s
  Running `target/debug/examples/02_if_else`
number=5
❖ khoi@khoi-Vostro-3520:~/Documents/TutorialRust$
```

Vòng lặp (Loops)

Rust có 3 loại vòng lặp: loop, while và for

- **loop**: Vòng lặp vô tận (cho đến khi bạn gọi break). Thường được sử dụng trong các dự án lập trình vi điều khiển.

```
fn main() {  
    let mut count = 0;  
    loop {  
        count += 1;  
        if count == 5 { break; }  
    }  
    println!("count={count}");  
}
```



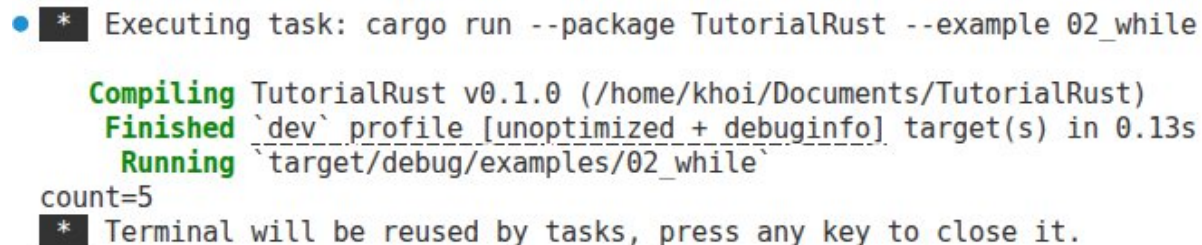
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

• * Executing task: cargo run --package TutorialRust --example 02_loop

Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
Running `target/debug/examples/02\_loop`
count=5

- **while**: Lặp khi điều kiện còn đúng.

```
fn main() {  
    let mut count = 0;  
    while count != 5 {  
        count += 1;  
    }  
    println!("count={count}");  
}
```



• * Executing task: cargo run --package TutorialRust --example 02_while

Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s
Running `target/debug/examples/02\_while`
count=5
* Terminal will be reused by tasks, press any key to close it.

- **for**: Phổ biến và an toàn nhất, dùng để duyệt qua một tập hợp.

```
fn main() {  
    // Lưu ý: Vòng lặp for sẽ duyệt từ 1 đến 4 (không tính 5)  
    for number in 1..5 {
```

```
        println!("{number}!");  
    }  
}
```

● * Executing task: cargo run --package TutorialRust --example 02_for

Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)

Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s

Running `target/debug/examples/02\_for`

1!

2!

3!

4!

* Terminal will be reused by tasks, press any key to close it.

NGÀY 3: SỞ HỮU (OWNERSHIP) VÀ MƯỢN (BORROWING)

Đây là cơ chế giúp Rust quản lý bộ nhớ an toàn tuyệt đối mà không cần đến Bộ thu gom rác (Garbage Collector) như Go, Java hay Python.

Bài 1: Hệ thống sở hữu (Ownership) là gì?

Để quản lý bộ nhớ máy tính (phần RAM được cấp phát cho chương trình), các ngôn ngữ thường chọn 1 trong 2 cách:

1. **Dọn dẹp thủ công (C/C++):** Lập trình viên tự cấp phát và tự xóa. Quên xóa sẽ gây rò rỉ bộ nhớ (memory leak), xóa nhầm sẽ gây sập chương trình.
2. **Bộ thu gom rác (Java, Python, Go):** Một chương trình chạy ngầm đi quét và dọn rác giúp bạn. An toàn nhưng làm chậm hệ thống.

Rust chọn lối đi thứ ba, sử dụng **3 quy tắc sở hữu** được kiểm tra ngay khi biên dịch:

- Mỗi giá trị trong Rust đều được có một biến làm chủ của nó (gọi là owner).
- Tại một thời điểm, chỉ có duy nhất một owner.
- Khi owner đi ra khỏi phạm vi hoạt động (out of scope), giá trị đó sẽ tự động bị xóa khỏi bộ nhớ.

Ví dụ về Scope (Phạm vi):

```
fn main() {  
    {  
        let s = "hello";  
        println!("{}", s);  
    }  
    println!("{}", s);  
}
```

// Biến s chưa xuất hiện ở đây
// s bắt đầu có hiệu lực kể từ dòng này

// s "bốc hơi" khỏi bộ nhớ

❌ * Executing task: cargo run --package TutorialRust --example 03_owner

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)  
error[E0425]: cannot find value `s` in this scope  
--> examples/03_owner.rs:8:16  
  
8 |     println!("{}", s);  
  |                   ^  
  
help: the binding `s` is available in a different scope in the same function  
--> examples/03_owner.rs:5:13  
  
5 |         let s = "hello";  
  |         ^
```

For more information about this error, try `rustc --explain E0425`.

error: could not compile `TutorialRust` (example "03_owner") due to 1 previous error

* The terminal process "cargo 'run', '--package', 'TutorialRust', '--example', '03_owner'" failed to launch (exit code: 101).

* Terminal will be reused by tasks, press any key to close it.

Bài 2: Hành vi dịch chuyển (Move) trong Heap

Hãy xem chuyện gì xảy ra với kiểu dữ liệu String (kiểu chuỗi có thể co giãn kích thước, được lưu trên vùng nhớ Heap):

```
fn main() {  
    let s1 = String::from("hello");  
    let s2 = s1;  
  
    println!("{s1}");  
}
```

❶ * Executing task: cargo run --package TutorialRust --example 03_string

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)  
error[E0382]: borrow of moved value: `s1`  
--> examples/03_string.rs:7:16  
4 |     let s1 = String::from("hello");  
  |     -- move occurs because `s1` has type `String`, which does not implement the `Copy` trait  
5 |     let s2 = s1;  
  |           -- value moved here  
6 |  
7 |     println!("{s1}");  
  |               ^^ value borrowed here after move  
  
help: consider cloning the value if the performance cost is acceptable  
5 |     let s2 = s1.clone();  
  |               +++++++  
  
warning: unused variable: `s2`  
--> examples/03_string.rs:5:9  
5 |     let s2 = s1;  
  |     ^^ help: if this is intentional, prefix it with an underscore: `_s2`  
  
= note: `#[warn(unused_variables)]` (part of `#[warn(unused)]`) on by default
```

Giải thích: Trong các ngôn ngữ khác, `s2 = s1` nghĩa là cả hai cùng trỏ chung vào một vùng nhớ. Nhưng với Rust, vì quy tắc “chỉ có duy nhất một owner”, quyền sở hữu vùng nhớ “hello” đã bị dịch chuyển từ `s1` sang `s2`. Kể từ khoảnh khắc đó, `s1` bị coi là “vô giá trị” và không thể sử dụng được nữa.

Bài 3: Mượn dữ liệu (Borrowing) và Tham chiếu (References)

Nếu cứ mỗi lần gán dữ liệu hoặc truyền biến vào hàm mà dữ liệu lại bị “mất quyền sở hữu” thì quá phiền phức. Để giải quyết, Rust cho phép chúng ta Mượn (Borrowing) bằng cách sử dụng ký tự `&` (gọi là tham chiếu - reference).

Mượn bất biến (Immutable Reference)

```
fn main() {  
    let s1 = String::from("hello");  
  
    let len = count_len(&s1);  
}
```



```

    let s2 = &s1;

    println!("String '{}' has length {}", s1, len);
    println!("String s2 = {s2}");
}

fn count_len(s: &String) -> usize {
    s.len()
}

```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s
Running `target/debug/examples/03\_immut\_ref`
String 'hello' has length 5
String s2 = hello
* Terminal will be reused by tasks, press any key to close it.

Mượn có thể thay đổi (Mutable Reference)

```

fn main() {
    let mut s = String::from("hello world");

    upper_str(&mut s);
    println!("{s}");
}

fn upper_str(str: &mut String) {
    str.make_ascii_uppercase();
}

```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `target/debug/examples/03\_mut\_ref`
HELLO WORLD
* Terminal will be reused by tasks, press any key to close it.

NGÀY 4: TỔ CHỨC DỮ LIỆU VỚI STRUCT VÀ ENUM

Bài 1: Struct

Struct giống như một chiếc hộp chứa nhiều ngăn, mỗi ngăn lưu trữ một mẫu dữ liệu có tên và kiểu dữ liệu riêng (gọi là các trường - fields). Nếu bạn từng học C/C++ hay OOP, Struct trong Rust tương tự như thuộc tính của một Class.

1.1 Khai báo và sử dụng Struct

Hãy tạo một kiểu dữ liệu đại diện cho một người dùng (User):

```
struct User {
    username: String,
    sign_in_count: u64,
}

fn main() {
    let mut user1 = User {
        username: String::from("Nguyen Van A"),
        sign_in_count: 1,
    };

    user1.sign_in_count = 2;
    println!("User {} signed in {} times.", user1.username,
user1.sign_in_count);
}
```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `target/debug/examples/04\_struct`

User Nguyen Van A signed in 2 times.

* Terminal will be reused by tasks, press any key to close it.

1.2 impl - Hành động cho Struct

Rust không có Class, nhưng bạn có thể định nghĩa các Phương thức (Methods) cho Struct bằng cách sử dụng từ khóa impl (Implementation).

```
struct Rectangle {
    width: u32,
    length: u32,
}

impl Rectangle {
```

```

fn area(&self) -> u32 {
    self.width * self.length
}

fn set_width(&mut self, new_width: u32) {
    self.width = new_width;
}

fn set_length(&mut self, new_length: u32) {
    self.length = new_length;
}

fn new(width: u32, length: u32) -> Rectangle {
    Rectangle { width, length }
}

fn main() {
    let mut rec = Rectangle { width: 30, length: 40};
    println!("Area of rectangle is {}", rec.area());

    rec.set_length(100);
    rec.set_width(500);
    println!("Area of rectangle is {}", rec.area());

    let other_rec = Rectangle::new(10, 40);
    println!("Area of rectangle is {}", other_rec.area());
}

```

```

• Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s
  Running `target/debug/examples/04_impl`
Area of rectangle is 1200
Area of rectangle is 50000
Area of rectangle is 400
* Terminal will be reused by tasks, press any key to close it.

```

1.2.1 Phương thức (Methods) - Có self

Cách self nhận quyền sở hữu quyết định cách phương thức đó tác động lên dữ liệu:

- **&self (Mượn bất biến):** Dùng nhiều nhất (chiếm 90%). Phương thức chỉ đọc dữ liệu của struct chứ không thay đổi nó.
- **&mut self (Mượn có thể thay đổi):** Dùng khi phương thức cần chỉnh sửa các giá trị bên trong struct.

- **self (Lấy luôn quyền sở hữu):** Rất hiếm dùng. Khi gọi phương thức này, thực thể gốc sẽ bị hủy (Move) ngay sau đó. Thường dùng cho các hàm tiêu hủy hoặc chuyển đổi dữ liệu (tương tự như hàm “consume”).

1.2.2 Hàm liên kết (Associated Functions) - Không có self

Đây là hàm nằm trong khối impl nhưng không nhận self làm tham số. Vì không có self, chúng không thuộc về một thực thể nào cụ thể, mà thuộc về chính “tên kiểu dữ liệu” đó. Nó tương đương với **Static Method** trong các ngôn ngữ khác.

Bài 2: Enum

Nếu Struct giúp bạn gom nhóm các dữ liệu cùng tồn tại, thì Enum cho phép bạn định nghĩa một kiểu dữ liệu mà giá trị của nó chỉ có thể là một trong các khả năng được liệt kê.

2.1 Khai báo Enum cơ bản

```
#[derive(Debug)]
enum Status {
    Processing,
    Delivering,
    Delivered,
    Cancelled,
}

fn main() {
    let mut packet_status = Status::Processing;

    println!("Your packet is {:?}", packet_status);

    packet_status = Status::Cancelled;

    println!("Your packet is {:?}", packet_status);
}
```

```
warning: `TutorialRust` (example "04_enum") generated 1 warning
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
  Running `target/debug/examples/04_enum`
Your packet is Processing
Your packet is Cancelled
* Terminal will be reused by tasks, press any key to close it.
```

Lưu ý: Kiểu dữ liệu của enum không được đại diện bởi các con số ẩn (0, 1, 2, 3,...) như các C++, Java hay các ngôn ngữ khác, do đó, để có thể debug đối với enum thì cần phải thêm dòng `#[derive(Debug)]` trước khi khai báo enum để có thể in ra giá trị của chúng.

2.2 Sức mạnh vượt trội: Enum chứa dữ liệu đi kèm (Data-bearing Enums)

Trong Rust, mỗi nhánh của Enum có thể đính kèm dữ liệu với các kiểu dữ liệu khác nhau. Đây chính là sức mạnh của Rust giúp code cực kỳ gọn gàng.

Ta hãy cùng nâng cấp đoạn code trên:

```
#[derive(Debug)]
enum Status {
    Processing,
    Delivering(String),
    Delivered,
    Cancelled{ reason: String, code: u32 },
}

fn main() {
    let packet1 = Status::Delivering(String::from("J&T Express"));
    let packet2 = Status::Cancelled { reason: String::from("Out of stock"), code: 404 };

    println!("{:?}", packet1);
    println!("{:?}", packet2);
}
```

```
warning: `TutorialRust` (example "04_enums_advance") generated 3 warnings
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
  Running `target/debug/examples/04_enums_advance`
Delivering("J&T Express")
Cancelled { reason: "Out of stock", code: 404 }
* Terminal will be reused by tasks, press any key to close it.
```

2.3 Khớp mẫu với match (Pattern Matching)

Để sử dụng hoặc lấy dữ liệu nằm bên trong các nhánh của Enum, Rust không dùng lệnh `i/else` thông thường mà bắt buộc phải dùng cấu trúc khớp mẫu (`match`).

Điểm hay của `match` là trình biên dịch sẽ kiểm tra xem bạn đã xử lý hết tất cả các nhánh chưa. Nếu bạn bỏ sót một trường hợp, Rust sẽ báo lỗi ngay lập tức khi bạn bấm dịch.

```
#[derive(Debug)]
enum Status {
    Processing,
    Delivering(String),
    Delivered,
    Cancelled{ reason: String, code: u32 },
}

fn get_status_message(packet: Status) {
    match packet {
        Status::Processing => {
```

```

        println!("Shop is packet your packet. Please waiting...");
    }

    Status::Delivering(shipper) => {
        println!("Your packet is being shipped by: {shipper}");
    }

    Status::Delivered => {
        println!("Your packet was delivered sucessfully.");
    }

    Status::Cancelled { reason, code } => {
        println!("Your packet was cancelled. Reason: {reason}. Code:
{code}");
    }
}

fn main() {
    let packet1 = Status::Delivering(String::from("J&T Express"));
    let packet2 = Status::Cancelled { reason: String::from("Out of
stock"), code: 404 };

    get_status_message(packet1);
    get_status_message(packet2);
}

```

```

warning: `TutorialRust` (example "04_enums_advance") generated 1 warning
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `target/debug/examples/04_enums_advance`
Your packet is being shipped by: J&T Express
Your packet was cancelled. Reason: Out of stock. Code: 404
* Terminal will be reused by tasks, press any key to close it.

```

2.4 Option Enum - Xóa số lỗi Null

Trong các ngôn ngữ khác, một biến có thể bị rỗng (null hoặc undefined). Nếu bạn quên kiểm tra mà vẫn mang đi sử dụng, chương trình sẽ bị sập.

Rust không có null. Để biểu diễn một giá trị "có thể có hoặc có thể không", Rust dùng Option như một hướng đi an toàn.

Bản chất của Option là một Enum có 2 nhánh:

- Some(...): Chiếc hộp có chứa đồ bên trong.
- None: Chiếc hộp trống rỗng (thay thế cho null).

Ví dụ: Khi bạn mở một hộp quà:

```
fn main() {
    let gifted_box: Option<&str> = Some("TV");
    let empty_box: Option<&str> = None;

    match gifted_box {
        Some(stuff) => println!("Congratulation! You received:
{stuff}"),
        None => println!("Sorry, this is an empty box"),
    }

    // empty_box.unwrap(); // Không nên vì nếu hộp rỗng (None) sẽ gây
crash.

    let other_stuff = empty_box.unwrap_or("discount 5%");
    println!("Your box is empty but you will have {other_stuff}.");

    if let Some(stuff) = gifted_box {
        println!("Your gift is: {stuff}");
    }
}
```

```
• Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
  Running `target/debug/examples/04_option`
Congratulation! You received: TV
Your box is empty but you will have discount 5%.
Your gift is: TV
* Terminal will be reused by tasks, press any key to close it.
```

Giải thích: Thay vì viết match dài dòng, bạn có thể dùng 3 cách ngắn gọn sau:

Lệnh mở hộp	Cách hoạt động	Rủi ro
.unwrap()	Giật phăng nắp hộp, lấy đồ bên trong ra ngay	Nguy hiểm! Nếu hộp rỗng, chương trình sẽ crash ngay lập tức.
.unwrap_or(default: <i>đồ_dự_phòng</i>)	Mở hộp lấy đồ. Nếu hộp rỗng, tự động lấy món đồ dự phòng thay thế.	An toàn tuyệt đối
if let Some(<i>đồ</i>) = <i>hộp</i>	Cách viết tắt cho “Nếu trong hộp có đồ, thì làm việc này	An toàn. Bỏ qua trường hợp rỗng.

NGÀY 5: TẬP HỢP DỮ LIỆU - VECTOR VÀ HASHMAP

Bài 1: Vector - Mảng động cơ giãn tùy ý

Vector là một danh sách các phần tử cùng kiểu dữ liệu, được lưu trữ liên tiếp nhau trên vùng nhớ Heap. Nó có thể tự động phình to ra khi bạn thêm phần tử mới.

Khởi tạo và thêm phần tử

Có hai cách tạo vector:

- Dùng hàm `Vec::new()`
- Dùng macro `vec!` cho nhanh

```
fn main() {  
    let mut v1: Vec<i32> = Vec::new();  
    v1.push(1); // Thêm phần tử vào cuối  
    v1.push(2);  
  
    let mut v2 = vec![10, 20, 30];  
    v2.pop();  
    v2.push(40);  
  
    println!("v1={:?}", v1);  
    println!("v2={:?}", v2);  
}
```

```
• Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)  
  Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s  
   Running `target/debug/examples/05_vector`  
v1=[1, 2]  
v2=[10, 20, 40]  
* Terminal will be reused by tasks, press any key to close it.
```

Đọc phần tử

Để lấy một phần tử theo chỉ số (index), Rust cung cấp 2 cách:

```
fn main() {  
    let v = vec![10, 20, 30];  
  
    // Cách 1: Dùng dấu ngoặc vuông (Nguy hiểm!)  
    let element = v[0]; // Nếu vị trí vượt ra ngoài size -> crash  
    println!("Element: {element}");  
}
```

```
// Cách 2: Dùng .get() (An toàn - Trả về một Option)
match v.get(100) {
    Some(value) => println!("Element: {value}"),
    None => println!("Index is out of range!"),
}
}
```

• * Executing task: cargo run --package TutorialRust --example 05_read_vector

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.15s
Running `target/debug/examples/05_read_vector`
Element: 10
Index is out of range!
* Terminal will be reused by tasks, press any key to close it.
```

Bài 2: HashMap

Nếu vector tìm kiếm bằng số thứ tự, thì HashMap lưu trữ dữ liệu theo cặp Key - Value.

Khởi tạo và sử dụng HashMap

```
use std::collections::HashMap;

fn main() {
    let mut point = HashMap::new();

    point.insert(String::from("Blue"), 10);
    point.insert(String::from("Yellow"), 50);

    let team_name = String::from("Blue");
    match point.get(&team_name) {
        Some(point) => println!("Team {team_name} has: {point}
points."),
        None => println!("Team {team_name} not found.")
    }
}
```

• **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.18s
Running `target/debug/examples/05\_hashmap`
Team Blue has: 10 points.
* Terminal will be reused by tasks, press any key to close it.

Duyệt qua các tập hợp dữ liệu

```
fn main() {
    let mut menu = HashMap::new();
```

```
menu.insert("Tea", 15000);  
menu.insert("Coffee", 20000);  
menu.insert("Matcha Latte", 50000);  
menu.insert("Orange Juice", 30000);  
  
for (drink, cost) in &menu {  
    println!("{drink} costs {cost} VND");  
}
```

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
 Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
 Running `target/debug/examples/05\_hashmap\_scan`
Coffee costs 20000 VND
Tea costs 15000 VND
Matcha Latte costs 50000 VND
Orange Juice costs 30000 VND
* Terminal will be reused by tasks, press any key to close it.

NGÀY 6: XỬ LÝ LỖI

Rust không sử dụng cơ chế try/catch giống như Java, C# hay Python. Thay vào đó, nó chia lỗi thành 2 loại rõ ràng: **Lỗi không thể phục hồi** và **Lỗi có thể phục hồi**.

Bài 1: Lỗi không thể phục hồi với *panic!*

Đây là lỗi nghiêm trọng khiến chương trình của bạn buộc phải dừng lại ngay lập tức vì không thể tiếp tục chạy được nữa (ví dụ: truy cập mảng vượt quá chỉ số, hoặc máy tính hết bộ nhớ).

```
fn main() {  
    println!("This line is still running...");  
  
    panic!("Overheated. Terminating...");  
  
    println!("This line won't run...");  
}
```

```
warning: `TutorialRust` (example "06_panic") generated 1 warning  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.13s  
Running `target/debug/examples/06_panic`  
This line is still running...
```

```
thread 'main' (31527) panicked at examples/06_panic.rs:6:5:  
Overheated. Terminating...
```

Bài 2: Lỗi có thể phục hồi với Result Enum

Trong thực tế, đa số lỗi thuộc loại có thể tha thứ và sửa sai được. Ví dụ: Bạn mở một file nhưng không thấy file đó đâu. Thay vì làm sập cả hệ thống, bạn chỉ cần báo cho người dùng biết để họ chọn file khác hoặc tạo file mới.

Để xử lý việc này, Rust sử dụng một Enum có sẵn rất thông minh tên là Result:

```
// Định nghĩa của Rust trong thư viện chuẩn:  
enum Result<T, E> {  
    Ok(T), // Thành công: Trả về dữ liệu kiểu T  
    Err(E), // Thất bại: Trả về thông tin lỗi kiểu E  
}
```

Nó có cơ chế hoạt động giống hệt như hộp quà Option ở Ngày 4, nhưng thay vì trả về None khi trống rỗng, nó trả về Err(E).

Ví dụ: Phép chia cho 0

```
fn divide(dividend: f32, divisor: f32) -> Result<f32, String> {  
    if divisor == 0.0 {  
        Err(String::from("Cannot divide by 0!"))  
    } else {
```

```

        Ok(dividend / divisor)
    }
}

fn main() {
    let result = divide(10.0, 0.0);
    match result {
        Ok(quotient) => println!("Result: {quotient}"),
        Err(error) => println!("Failed! Reason: {error}"),
    }
}

```

- Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
Running `target/debug/examples/06\_result`
 Failed! Reason: Cannot divide by 0!
 * Terminal will be reused by tasks, press any key to close it.

Tuy nhiên, nhìn ví dụ trên khá dài và khó quản lý. Do đó, Rust hỗ trợ truyền lỗi nhanh với toán tử “?”

Ví dụ:

```

fn calculate_per_person_cost(total_cost: f32, people_count: f32) ->
Result<f32, String> {
    if people_count == 0.0 {
        Err(String::from("Cannot divide by 0! Group must have at least 1
person."))
    } else {
        Ok(total_cost / people_count)
    }
}

fn generate_invoice(total_cost: f32, people_count: f32) ->
Result<String, String> {
    let cost = calculate_per_person_cost(total_cost, people_count)?;
    Ok(format!("Invoice created. Cost per person: ${cost}"))
}

fn main() {
    let booking_result = generate_invoice(5000.0, 0.0);
    match booking_result {
        Ok(invoice) => println!("{invoice}"),
        Err(error) => println!("Booking Error: {error}"),
    }
}

```


- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.14s
Running `target/debug/examples/06\_result\_question\_mark`
Booking Error: Cannot divide by 0! Group must have at least 1 person.
* Terminal will be reused by tasks, press any key to close it.

Hoặc trường hợp: `let booking_result = generate_invoice(5000.0, 10.0);`

- **Compiling** TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
Running `target/debug/examples/06\_result\_question\_mark`
Invoice created. Cost per person: \$500
* Terminal will be reused by tasks, press any key to close it.

NGÀY 7: TRAITS VÀ GENERICS - TÁI SỬ DỤNG MÃ NGUỒN

Bài 1: Generics - Kiểu dữ liệu tổng quát

```
struct Point<T> {  
    x: T,  
    y: T,  
}  
  
fn main() {  
    let point1 = Point { x: 5, y: 10 };  
    let point2 = Point { x: 1.5, y: 10.5 };  
}
```

Bài 2: Traits - Định nghĩa hành vi chung (Interfaces)

Trait trong Rust tương tự như Interface ở các ngôn ngữ khác. Nó là một bản giao ước định nghĩa các hành động (hàm) mà một kiểu dữ liệu bắt buộc phải có nếu muốn tham gia hệ thống.

Ví dụ:

```
pub trait Summary {  
    fn summarize(&self) -> String;  
}  
  
pub struct NewsArticle {  
    pub headline: String,  
    pub content: String,  
}  
  
pub struct Tweet {  
    pub username: String,  
    pub content: String,  
}  
  
impl Summary for NewsArticle {  
    fn summarize(&self) -> String {  
        format!("News: {}: {}", self.headline, self.content)  
    }  
}  
  
impl Summary for Tweet {  
    fn summarize(&self) -> String {
```

```

        format!("Tweet: {}: {}", self.username, self.content)
    }
}

fn main() {
    let tweet = Tweet {
        username: String::from("rust_lang"),
        content: String::from("Thanks for being here!"),
    };

    let news = NewsArticle {
        headline: String::from("Rust lang is awesome"),
        content: String::from("Rust is a general-purpose programming
language without relying on a garbage collector."),
    };

    println!("{}", tweet.summarize());
    println!("{}", news.summarize());
}

```

```

• Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.11s
Running `target/debug/examples/07_trait`
Tweet: rust_lang: Thanks for being here!
News: Rust lang is awesome: Rust is a general-purpose programming language without rel
ying on a garbage collector.
* Terminal will be reused by tasks, press any key to close it.

```

Bài 3: Trait Bounds - Sự kết hợp hoàn hảo

Sức mạnh khi kết hợp Generics và Traits.

```

struct Point<T> {
    x: T,
    y: T,
}

impl<T: std::fmt::Display> Point<T> {
    fn display(&self) {
        println!("Point [{} {}]", self.x, self.y);
    }
}

fn main() {
    let point1 = Point { x: 5, y: 10 };
    let point2 = Point { x: 1.5, y: 10.5 };
}

```

```
point1.display();  
point2.display();  
}
```

-  Executing task: cargo run --package TutorialRust --example 07_generic

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.18s  
Running `target/debug/examples/07_generic`
```

Point [5 10]

Point [1.5 10.5]

 Terminal will be reused by tasks, press any key to close it.

NGÀY 8: LIFETIMES - QUẢN LÝ VÒNG ĐỜI CỦA THAM CHIẾU

Bài 1: Bản chất của Lifetimes là gì?

Mọi biến số và tham chiếu trong Rust đều có một vòng đời - tức là phạm vi code mà nó tồn tại hợp lệ trong bộ nhớ RAM từ lúc sinh ra đến lúc bị hủy.

Thông thường, trình biên dịch Rust sở hữu một bộ kiểm soát mượn dữ liệu cực kỳ nghiêm khắc gọi là Borrow Checker. Nó sử dụng tính toán vòng đời để đảm bảo không một tham chiếu nào được phép sống lâu hơn thực thể gốc mà nó đang trỏ tới.

Ví dụ:

```
fn main() {  
    let r;                                // -----+-- Vòng đời của r (Scope 'a')  
                                        //          |  
    {                                    //          |  
        let x = 5;                       // ----+   | Vòng đời của x (Scope 'b')  
        r = &x;                           //      |   |  
    }                                     // ----+   | x bị hủy tại đây!  
                                        //          |  
    println!("r: {r}");                  //          | LỖI! r cố tình in giá trị của x  
    // đã bị hủy!  
}
```

Kết quả: Rust sẽ từ chối biên dịch đoạn code này ngay lập tức vì vòng đời của x (Scope 'b') ngắn hơn vòng đời của r (Scope 'a'). Bạn không thể giữ một tham chiếu đến một thứ không còn tồn tại.

Bài 2: Ký pháp Lifetimes (Lifetime Annotations)

Trong hầu hết các hàm bình thường, bạn không cần phải viết gì cả vì Rust tự hiểu. Nhưng khi bạn viết một hàm nhận vào nhiều tham chiếu và trả về một tham chiếu, Rust sẽ bối rối không biết cái tham chiếu trả về kia ăn theo vòng đời của tham chiếu đầu vào nào.

Lúc này, bạn phải dùng ký pháp Lifetimes (bắt đầu bằng dấu nháy đơn và một chữ cái thường, ví dụ: 'a, 'b)

Cấp độ 1: Vòng đời ẩn (Lifetime Elision)

```
fn print_msg(s: &str) { println!("{}", s); }
```

Khi này Rust sẽ ngầm hiểu là: `fn print_msg(<'a>(s: &'a str)`

Cấp độ 2: Một lifetimes duy nhất (<'a>)

Dùng khi hàm nhận vào nhiều tham chiếu và trả về một tham chiếu, khi này bắt buộc phải khai báo lifetimes.

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}
```

Cấp 3: Nhiều lifetime độc lập (<'a, 'b>)

Dùng khi hàm nhận vào nhiều tham chiếu độc lập và tham chiếu trả về chỉ ăn theo thời gian sống của một nguồn cụ thể. việc tách riêng giúp nguồn còn lại có thể bị hủy sớm mà không làm sập chương trình.

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {  
    if x.len() > y.len() { x } else { y }  
}
```

Cấp 4: Vòng đời vĩnh cửu ('static)

Dữ liệu mang lifetime 'static sẽ tồn tại suốt toàn bộ thời gian chạy của phần mềm (được lưu thẳng vào phân vùng mã máy của file thực thi).

```
let config_str: &'static str = "BAUD_RATE:9600";
```

Bài 2: Dự án nhỏ với Bộ giải mã gói tin IoT siêu tốc (Zero-Copy Packet Parser)

```
struct PacketParser<'a> {  
    raw_payload: &'a str,  
}  
  
impl <'a> PacketParser<'a> {  
    fn extract_value<'b>(&self, filter: &'b str) -> &'a str {  
        // Hàm trích xuất giá trị dựa trên một từ khóa tạm thời  
(filter_key)  
        if let Some(pos) = self.raw_payload.find(filter) {  
            let start_idx = pos + filter.len();  
            &self.raw_payload[start_idx..]  
        } else {  
            "NOT_FOUND"  
        }  
    }  
}
```



```
fn main() {
    let uart_buffer = String::from("DEVICE:ROBOT_01:BAUD:9600");
    let parser = PacketParser { raw_payload: &uart_buffer };

    let baud_rate;
    {
        let filter_key = String::from("BAUD:");

        baud_rate = parser.extract_value(&filter_key);
    }

    println!("Baudrate: {baud_rate}");
}
```

• * Executing task: cargo run --package TutorialRust --example 10_small_prj

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.16s
Running `target/debug/examples/10_small_prj`
```

Baudrate: 9600

* Terminal will be reused by tasks, press any key to close it.

- Zero-Copy: Biến `baud_rate` bản chất chỉ là một con trỏ trỏ trực tiếp vào phân đoạn nằm trong `uart_buffer`. Chúng ta không hề tốn thêm một byte RAM nào để tạo ra chuỗi mới. Đây là lí do tại sao Rust chạy cực nhẹ trên vi điều khiển.
- Nếu bạn vô tình viết code xóa biến `uart_buffer` trước dòng `println!`, Borrow Checker sẽ khóa ngay lập tức và báo lỗi khi biên dịch.

NGÀY 9: LOGGING

Trong Rust, hệ thống logging được chia làm 2 phần độc lập: **Mặt nạ giao diện (Facade API)** và **Lỗi thực thi (Loggers)**.

Mọi dự án Rust muốn dùng log đều phải cài đặt thư viện (crate) mang tên log. Đây là thư viện chính thức do đội ngũ Rust phát triển, cung cấp 5 cấp độ log từ nhẹ đến nặng:

- **error!** : Lỗi nghiêm trọng khiến một tính năng hoặc hệ thống bị sập (ví dụ: sập Database).
- **warn!** : Cảnh báo các nguy cơ tiềm ẩn (ví dụ: bộ nhớ RAM sắp đầy, kết nối mạng chậm).
- **info!** : Thông tin hoạt động thông thường (ví dụ: khởi chạy phần mềm, user đăng nhập thành công).
- **debug!** : Thông tin chi tiết phục vụ cho việc tìm vết và sửa lỗi trong quá trình phát triển.
- **trace!** : Thông tin siêu chi tiết, ghi lại từng bước nhỏ nhất (ví dụ: từng byte dữ liệu chạy qua mạng).

Cấu hình: log + env_logger

Bước 1: Khai báo thư viện trong Cargo.toml

Thêm 2 thư viện này vào mục [dependencies] trong file Cargo.toml gốc của dự án.

Gõ lệnh sau vào Terminal để Cargo tự động thêm vào:

```
cargo add log env_logger
```

Trong file Cargo.toml sẽ được cấu hình như sau:

```
[dependencies]
log = "0.4"
env_logger = "0.11"
```

Bước 2: Viết mã nguồn

Ví dụ: Tạo một file ví dụ (examples/08_logging.rs):

```
use log::{error, warn, info, debug, trace};

fn main() {
    env_logger::init();

    error!("Database crashed!");
    warn!("Alert: RAM overhead 80%");
    info!("User 'admin' has been logged in");
}
```

```
    debug!("x=5");
    trace!("Receive 12 data bytes from IP...");
}
```

Chạy logging bằng lệnh

```
RUST_LOG=trace cargo run --example 08_logging
```

```
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ RUST_LOG=trace cargo run --example 08_logging
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
    Running `target/debug/examples/08_logging`
[2026-06-14T06:04:37Z ERROR 08_logging] Database crashed!
[2026-06-14T06:04:37Z WARN 08_logging] Alert: RAM overhead 80%
[2026-06-14T06:04:37Z INFO 08_logging] User 'admin' has been logged in
[2026-06-14T06:04:37Z DEBUG 08_logging] x=5
[2026-06-14T06:04:37Z TRACE 08_logging] Receive 12 data bytes from IP...
○ khoi@khoi-Vostro-3520:~/Documents/TutorialRust$
```

Hoặc chỉ xem mức thông tin thông thường (INFO) trở lên:

```
RUST_LOG=info cargo run --example 08_logging
```

```
• khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ RUST_LOG=info cargo run --example 08_logging
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.03s
    Running `target/debug/examples/08_logging`
[2026-06-14T06:14:31Z ERROR 08_logging] Database crashed!
[2026-06-14T06:14:31Z WARN 08_logging] Alert: RAM overhead 80%
[2026-06-14T06:14:31Z INFO 08_logging] User 'admin' has been logged in
○ khoi@khoi-Vostro-3520:~/Documents/TutorialRust$
```

NGÀY 10: ĐỌC VÀ GHI FILE

Bài 1: Ghi dữ liệu xuống file

Để ghi dữ liệu vào một file văn bản, Rust cung cấp module `std::fs::File` và macro `writeln!`

Ví dụ:

```
use std::fs::File;
use std::io::Write;

fn main() -> std::io::Result<()> {
    // Hàm trả về một Result, ta dùng dấu '?' để truyền lỗi nhanh nếu
    // không tạo được file.
    let mut file = File::create("examples/logging.txt");

    writeln!(file, "First line.");

    Ok(())
}
```



Bài 2: Đọc dữ liệu từ file

Rust cung cấp nhiều cách đọc file tùy thuộc vào kích thước dữ liệu. Cách đơn giản nhất là đọc toàn bộ nội dung file thành một chuỗi String.

Ví dụ:

```
use std::fs;

fn main() -> std::io::Result<()> {
```

```
let content = fs::read_to_string("examples/data.txt");

println!("--- Data Read ---");
println!("{}", content);

Ok(())
}
```

examples > data.txt

```
1 username: ABC
2 password: 123
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SERIAL MONITOR

• * Executing task: cargo run --package TutorialRust --example 09_read_file

Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.12s
Running `target/debug/examples/09\_read\_file`

```
--- Data Read ---
username: ABC
password: 123
* Terminal will be reused by tasks, press any key to close it.
```

bash
Code
r... ✓
r... ✓

Bài 3: Ghi nối tiếp vào file

Trong thực tế, khi Robot chạy liên tục, dữ liệu cảm biến mới phải được ghi nối tiếp vào cuối file cũ chứ không được phép ghi đè. Để làm việc này, ta sử dụng `OpenOptions`.

Ví dụ:

```
use std::fs::OpenOptions;
use std::io::Write;

fn main() -> std::io::Result<()> {
    let mut file = OpenOptions::new()
        .write(true)
        .append(true)
        .open("examples/logging.txt")?;

    writeln!(file, "Second line.")?;

    Ok(())
}
```

```
}
```

examples > logging.txt

```
1 First line.  
2 Second line.  
3
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR

• * Executing task: cargo run --package TutorialRust --example 09_write_appending

```
Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)  
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.16s  
Running `target/debug/examples/09_write_appending`
```

* Terminal will be reused by tasks, press any key to close it.

NGÀY 11: QUẢN LÝ DỰ ÁN VÀ INCLUDE TRONG RUST

Trong C/C++, để chèn thư viện hoặc mã nguồn ta sẽ sử dụng câu lệnh `#include`. Tuy nhiên, Rust không sử dụng cơ chế đó. Thay vào đó, Rust quản lý dự án cực kỳ chặt chẽ bằng hệ thống Crates (Thư viện) và Modules (Mô-đun).

Bài 1: Include một thư viện ngoài

Crates là thuật ngữ để chỉ các gói thư viện trong Rust. Khi bạn muốn sử dụng một thư viện do cộng đồng viết sẵn (ví dụ: thư viện `log` để ghi nhật ký). Thực hiện qua 2 bước:

- **Bước 1:** Khai báo tên và phiên bản thư viện trong file cấu hình `Cargo.toml` gốc của dự án.
- **Bước 2:** sử dụng từ khóa `use` ở đầu file mã nguồn để đưa các hàm hoặc cấu trúc thư viện đó vào phạm vi sử dụng.

Ví dụ:

Bạn mở Terminal tại thư mục dự án và gõ lệnh để Cargo tự động thêm thư viện:

```
cargo add log
```

Lúc này, file `Cargo.toml` sẽ động xuất hiện dòng cấu hình (tham khảo):

```
[dependencies]
log = "0.4"
```

Trong file mã nguồn `main.rs`, bạn include tính năng bằng từ khóa `use`

```
use log::error;
fn main() {
    error!("WiFi disconnected!");
}
```

Bài 2: Include một file mã nguồn do bạn tự viết

Giả sử bạn có cấu trúc thư mục dự án như sau:

```
src/
├── main.rs
└── motor.rs <-- File bạn tự viết, chứa code điều khiển động cơ
```

Bước 1: Viết mã nguồn file con `src/motor.rs`

Để file khác có thể nhìn thấy và include được các hàm bên trong, bạn bắt buộc phải thêm từ khóa pub (public) trước tên hoặc cấu trúc:

```
// src/motor.rs

// Hàm public: cho phép bên ngoài truy cập
pub fn start_forward() {
    println!("Motors are spinning forward...");
}
```

Bước 2: Include file con vào file chính src/main.rs

Để liên kết file motor.rs vào main.rs, Rust sử dụng bộ đôi từ khóa: “mod” và “use”

```
// src/main.rs

// 1. KHAI BÁO: Báo cho trình biên dịch biết có một file tên là motor.rs
cùng cấp
mod motor;

// 2. SỬ DỤNG: "Include" hàm start_forward từ mô-đun motor vào
use motor::start_forward;

fn main() {
    // Gọi hàm điều khiển động cơ trực tiếp và an toàn
    start_forward();
}
```

NGÀY 12: NO_STD VÀ EMBEDDED RUST

Khi viết code Rust trên máy tính, hệ sinh thái sẽ mặc định nạp thư viện chuẩn std. Thư viện này phụ thuộc vào các tính năng của hệ điều hành (như Windows, Linux) để quản lý file, cấp phát bộ nhớ hay quản lý luồng.

Tuy nhiên trong thế giới của vi điều khiển, chip hoàn toàn không có hệ điều hành (bare-metal). Do đó, chúng ta phải ra lệnh cho Rust cắt bỏ toàn bộ những phần công kênh đó.

Bài 1: Khái niệm `#![no_std]`

Để báo cho trình biên dịch biết rằng đoạn code này sẽ được nạp thẳng xuống ROM của vi điều khiển, chúng ta sử dụng thuộc tính `#![no_std]` ở ngay dòng đầu tiên của file mã nguồn `main.rs`.

Khi bật, Rust sẽ:

- Loại bỏ bộ thu gom rác giúp code chạy với thời gian thực (real-time).
- Không có bộ nhớ Heap mặc định.
- Code dịch ra file `.bin` hoặc `.hex` chỉ vồn vện vài KB.

Bài 2: Framework một file code `#![no_std]` cơ bản

```
// 1. Khai báo môi trường không có hệ điều hành
#![no_std]
// 2. Báo cho Rust biết chúng ta tự quản lý điểm khởi chạy (Entry Point)
// của chip
#![no_std_main]

// Sử dụng một thư viện runtime phù hợp với kiến trúc chip (Ví dụ cho
// dòng ARM Cortex-M)
use cortex_m_rt::entry;
use panic_halt as _; // Nếu xảy ra panic (lỗi), lập tức treo chip để an
toàn phần cứng

// 3. Chỉ định đây là hàm sẽ chạy đầu tiên ngay khi cấp nguồn cho mạch
#[entry]
fn main() -> ! {
    // Logic cấu hình các chân PIN (GPIO), UART, SPI sẽ viết ở đây...

    // 4. Vòng lặp vô tận (Loop) - Trái tim của mọi hệ thống nhúng/Robot
    // Chip sẽ chạy vòng lặp này mãi mãi cho đến khi bị ngắt điện
    loop {
        // Đọc cảm biến, điều khiển động cơ, nhấp nháy LED
    }
}
```

NGÀY 13: COVERAGE TEST

Để kiểm tra Code Coverage (độ bao phủ mã nguồn) trong Rust, ta không sử dụng các công cụ thủ công mà tin dùng bộ công cụ tiêu chuẩn và mạnh mẽ nhất hiện nay: cargo-tarpaulin hoặc công cụ tích hợp sẵn của trình biên dịch là llvm-cov.

Bài 1: Các lệnh cơ bản

Đối với Linux

“tarpaulin” là một công cụ mở rộng của Cargo được thiết kế riêng cho Linux. Nó chạy cực nhanh và có thể xuất ra báo cáo trực quan ngay trên Terminal hoặc xuất ra file HTML.

Bước 1 (setup): Cài đặt

Chạy lệnh sau:

```
cargo install cargo-tarpaulin
```

Bước 2: Chạy kiểm tra Coverage

Tại thư mục gốc của dự án, hãy chạy lệnh

```
cargo tarpaulin
```

Bước 3: Xem kết quả

Màn hình Terminal sẽ quét qua toàn bộ các hàm test và in ra một bảng thống kê tỷ lệ phần trăm coverage

Ví dụ:

```
src/motor.rs: 85.0%  
src/sensor.rs: 40.0%  
src/main.rs: 100.0%  
-----  
Total Coverage: 75.3%
```

Bước 4 (optional): Nếu muốn xuất ra file HTML để click xem từng dòng code nào chưa được test, chạy lệnh sau:

```
cargo tarpaulin --out Html
```

Một file tarpaulin-report.html sẽ được sinh ra, bạn chỉ cần click đúp để mở nó bằng trình duyệt Web.

Đối với Windows hoặc macOS

tarpaulin không hỗ trợ tốt trên Windows và macOS do đó ta nên sử dụng bộ công cụ của chính trình biên dịch LLVM (cargo-llvm-cov).

Bước 1 (setup): Cài đặt công cụ và thành phần LLVM đi kèm

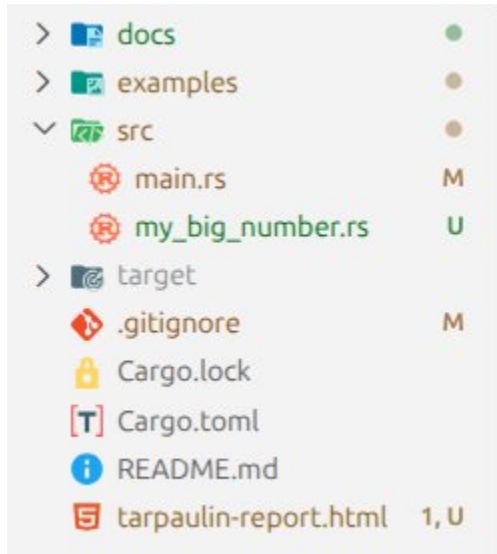
```
cargo install cargo-llvm-cov  
rustup component add llvm-tools-preview
```

Bước 2: Chạy lệnh quét Coverage và tự động mở trình duyệt xem báo cáo

```
cargo llvm-cov --html --open
```

ỨNG DỤNG VÀO BÀI TOÁN CỘNG 2 SỐ

Ta áp dụng coverage test vào một dự án thực tế: Cộng 2 chuỗi số lớn theo phương pháp tiểu học. Cấu trúc thư mục sẽ trông như sau:



Mã nguồn file my_big_number.rs

```
pub fn sum(a: &str, b: &str) -> String {
    if a.is_empty() || b.is_empty() {
        return String::from("0");
    } else {
        let chars_a: Vec<char> = a.chars().collect();
        let chars_b: Vec<char> = b.chars().collect();

        let len_a = chars_a.len();
        let len_b = chars_b.len();

        let max_len = std::cmp::max(len_a, len_b) + 1;

        let mut result_chars = vec!['0'; max_len];

        let mut i = (len_a as i32) - 1;
        let mut j = (len_b as i32) - 1;
        let mut carry = 0;

        for k in (0..max_len).rev() {
            if i < 0 && j < 0 && carry == 0 {
                break;
            }
        }
    }
}
```

```

        let mut digit_a = 0;
        let mut digit_b = 0;

        if i >= 0 {
            digit_a = chars_a[i as usize].to_digit(10).unwrap_or(0);
            i -= 1;
        }

        if j >= 0 {
            digit_b = chars_b[j as usize].to_digit(10).unwrap_or(0);
            j -= 1;
        }

        let total = digit_a + digit_b + carry;
        carry = total / 10;
        let current_digit = total % 10;

        result_chars[k] = std::char::from_digit(current_digit,
10).unwrap_or('0');
    }

    let mut result_str: String = result_chars.into_iter().collect();

    if result_str.starts_with('0') && result_str.len() > 1 {
        result_str.remove(0);
    }

    return result_str;
}
}

```

Giải thích:

- Dựa vào độ dài chuỗi a và b, hàm tính trước kích thước tối đa và khởi tạo sẵn mảng result_chars chứa toàn ký tự '0'. Việc này giúp nhét thẳng kết quả vào vị trí cuối lên đầu mà không tốn chi phí dịch chuyển dữ liệu trong RAM.
- Vòng lặp for chạy lùi (.rev()) từ hàng đơn vị về bên trái. Hai con trỏ i và j bóc tách từng chữ số để tính tổng: total = digit_a + digit_b + carry.
- Xử lý số 0 thừa: Vì chuỗi kết quả được cấp phát dư 1 vị trí để phòng trường hợp tràn hàng cao nhất (ví dụ với trường hợp "1" + "99" = "100") nên với các phép cộng không nhớ (ví dụ trường hợp "11" + "22" = "033") hàm sẽ tự động cắt bỏ ký tự '0' thừa ở đầu.

Mã nguồn file main.rs

```
mod my_big_number;
```

[illegible]

Giải thích: File main.rs là trung tâm của dự án, được chia làm hai chế độ hoạt động riêng biệt:

- Chế độ chạy thường (fn main): Kiểm tra thuật toán trong thực tế bằng cách truyền vào các chuỗi số thông thường và chuỗi siêu lớn.
- Chế độ kiểm thử (mod tests): Các hàm #[test] chỉ kích hoạt khi chạy lệnh cargo test hoặc quét coverage.

Kết quả chạy test:

cargo run

[illegible]

Chương trình chỉ chạy hàm main trong file main.rs.

cargo test

```
Finished `test` profile [unoptimized + debuginfo] target(s) in 0.12s
Running unittests src/main.rs (target/debug/deps/TutorialRust-92c86c2b8b39ce63)

running 3 tests
test tests::test_big_sum ... ok
test tests::test_edge_cases ... ok
test tests::test_mismatched_length_sum ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

cargo taurpaulin

```

khoi@khoi-Vostro-3520:~/Documents/TutorialRust$ cargo tarpaulin
2026-06-17T11:13:28.466064Z INFO cargo_tarpaulin::config: Creating config
2026-06-17T11:13:28.518119Z INFO cargo_tarpaulin: Running Tarpaulin
2026-06-17T11:13:28.518147Z INFO cargo_tarpaulin: Building project
2026-06-17T11:13:28.518242Z INFO cargo_tarpaulin::cargo: Cleaning project
    Compiling memchr v2.8.2
    Compiling regex-syntax v0.8.11
    Compiling utf8parse v0.2.2
    Compiling anstyle-query v1.1.5
    Compiling is_terminal_polyfill v1.70.2
    Compiling colorchoice v1.0.5
    Compiling log v0.4.32
    Compiling anstyle v1.0.14
    Compiling jiff v0.2.28
    Compiling anstyle-parse v1.0.0
    Compiling anstream v1.0.0
    Compiling aho-corasick v1.1.4
    Compiling regex-automata v0.4.14
    Compiling regex v1.12.4
    Compiling env_filter v1.0.1
    Compiling env_logger v0.11.10
    Compiling TutorialRust v0.1.0 (/home/khoi/Documents/TutorialRust)
    Finished `test` profile [unoptimized + debuginfo] target(s) in 4.68s
2026-06-17T11:13:33.338119Z INFO cargo_tarpaulin::process_handling::linux: Launching test
2026-06-17T11:13:33.338151Z INFO cargo_tarpaulin::process_handling: running /home/khoi/Documents/TutorialRust/target/debu
g/deps/TutorialRust-23aa15e96d9ecf1a

running 3 tests
test tests::test_mismatched_length_sum ... ok
test tests::test_edge_cases ... ok
test tests::test_big_sum ... ok

test result: ok. 3 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s

2026-06-17T11:13:33.534937Z INFO cargo_tarpaulin::report: Coverage Results:
|| Uncovered Lines:
|| src/main.rs: 5-8, 10-12
|| Tested/Total Lines:
|| src/main.rs: 0/7 +0.00%
|| src/my_big_number.rs: 30/30 +0.00%
||
81.08% coverage, 30/37 lines covered, +0.00% change in coverage

```

File HTML:

Note: Thư mục examples chứa các code ví dụ trong các ngày trước, không liên quan đến dự án hiện tại nên ta tập trung vào thư mục src.

Kết quả coverage của file main.rs:

Nhận xét: Khi chạy coverage test, Rust sẽ tập trung vào chạy “cargo test” thay vì “cargo run” do đó hàm main không chạy là bình thường. Để có thể kiểm tra các testcase đã bao quát hết các

trường hợp hay chưa, ta tập trung vào kết quả coverage của file my_big_number.rs sau:

Back /home/khoi/Documents/TutorialRust/src/my_big_number.rs		Covered: 30 of 30 (100.00%) 🌟
1	pub fn sum(a: &str, b: &str) -> String {	3✓
2	if a.is_empty() b.is_empty() {	3✓
3	return String::from("0");	1✓
4	} else {	
5	let chars_a: Vec<char> = a.chars().collect();	2✓
6	let chars_b: Vec<char> = b.chars().collect();	4✓
7		
8	let len_a = chars_a.len();	5✓
9	let len_b = chars_b.len();	1✓
10		
11	let max_len = std::cmp::max(len_a, len_b) + 1;	2✓
12		
13	let mut result_chars = vec!['0'; max_len];	3✓
14		
15	let mut i = (len_a as i32) - 1;	1✓
16	let mut j = (len_b as i32) - 1;	3✓
17	let mut carry = 0;	2✓
18		
19	for k in (0..max_len).rev() {	5✓
20	if i < 0 && j < 0 && carry == 0 {	2✓
21	break;	
22	}	
23		
24	let mut digit_a = 0;	2✓
25	let mut digit_b = 0;	1✓
26		
27	if i >= 0 {	3✓
28	digit_a = chars_a[i as usize].to_digit(10).unwrap_or(0);	1✓
29	i -= 1;	2✓
30	}	
31		
32	if j >= 0 {	3✓
33	digit_b = chars_b[j as usize].to_digit(10).unwrap_or(0);	1✓
34	j -= 1;	2✓
35	}	
36	let total = digit_a + digit_b + carry;	3✓
37	carry = total / 10;	3✓
38	let current_digit = total % 10;	2✓
39		
40	result_chars[k] = std::char::from_digit(current_digit, 10).unwrap_or('0');	1✓
41	}	
42		
43	let mut result_str: String = result_chars.into_iter().collect();	3✓

Nhận xét: Tất cả các line trong file my_big_number.rs đều được tô xanh chứng tỏ unit tests đã bao quát hết các trường hợp.